

CS 473 (Spring 2025)
Homework 10 (due May 1 Thu 10am)

Instructions: As in previous homeworks.

Problem 10.1: Consider the following problem: The input consists of n rectangles, where the i -th rectangle R_i has height $h_i \leq 1$, width $w_i \leq 1$, and area $a_i = h_i w_i$. The goal is to place the rectangles in a rectangular container of width 1 while minimizing the height of the container. Rotations of the rectangles are not allowed.

Consider the following greedy algorithm:

0. sort the rectangles so that $h_1 \geq h_2 \geq \dots \geq h_n$
1. $k_1 = 1$
2. for $m = 1, 2, \dots$ until $k_m = n + 1$ {
3. let k_{m+1} be the largest index such that $w_{k_m} + w_{k_m+1} + \dots + w_{k_{m+1}-1} \leq 1$
4. place $R_{k_m}, R_{k_m+1}, \dots, R_{k_{m+1}-1}$ in a new row (a “shelf”) of height h_{k_m}
- }

Show that this greedy algorithm runs in polynomial time and has approximation factor at most 3.

[Hint: Prove the inequality $a_{k_m} + a_{k_m+1} + \dots + a_{k_{m+1}} > h_{k_{m+1}}$. Then sum both sides...]

Solution:

Polynomial running time. Sorting the n rectangles by non-increasing height takes $O(n \log n)$ time. Then the single pass that scans through the list, forming each shelf by accumulating widths until they exceed 1, clearly takes $O(n)$ time. Hence the overall running time is $O(n \log n)$.

Approximation ratio. Let H_G be the total height of the packing produced by the greedy algorithm, and let H^* be the height of an optimal packing.

Label the shelves produced by the algorithm $m = 1, 2, \dots, M$, and for each m let k_{m+1} be the index of the first rectangle not placed on shelf m , so by construction

$$w_{k_m} + w_{k_m+1} + \dots + w_{k_{m+1}-1} \leq 1 \quad \text{but} \quad w_{k_m} + w_{k_m+1} + \dots + w_{k_{m+1}} > 1.$$

Each shelf m is given height h_{k_m} , so

$$H_G = \sum_{m=1}^M h_{k_m}.$$

For each shelf m ,

$$\sum_{i=k_m}^{k_{m+1}} a_i = \sum_{i=k_m}^{k_{m+1}} h_i w_i \geq h_{k_{m+1}} (w_{k_m} + \dots + w_{k_{m+1}}) > h_{k_{m+1}}.$$

Summing this strict inequality over $m = 1, 2, \dots, M$ gives

$$\sum_{m=1}^M \sum_{i=k_m}^{k_{m+1}} a_i > \sum_{m=1}^M h_{k_{m+1}} = \sum_{m=2}^{M+1} h_{k_m} = \sum_{m=2}^M h_{k_m}.$$

(Since $k_{M+1} = n + 1$ and we may set $h_{n+1} = 0$.) The left-hand side is

$$\sum_{m=1}^M \sum_{i=k_m}^{k_{m+1}} a_i = \sum_{m=1}^M \left(\sum_{i=k_m}^{k_{m+1}-1} a_i + a_{k_{m+1}} \right) = A + \sum_{m=1}^M a_{k_{m+1}} < 2A$$

and in any packing into a width-1 strip of height H^* the total area satisfies $A \leq H^*$. Hence

$$\sum_{m=2}^M h_{k_m} < 2A \leq 2H^*.$$

Therefore

$$H_G = \sum_{m=1}^M h_{k_m} = h_{k_1} + \sum_{m=2}^M h_{k_m} < h_{k_1} + 2H^*.$$

Finally, since the tallest rectangle has height h_{k_1} and must fit into any feasible packing, we have

$$h_{k_1} \leq H^*,$$

and thus

$$H_G < h_{k_1} + H^* \leq 3H^*.$$

The approximation factor is at most 3.

Problem 10.2: Consider the following variant of the traveling salesman problem: given a complete weighted graph $G = (V, E)$, where the weights satisfy $d(u, v) = d(v, u)$ and $d(u, v) \leq d(u, w) + d(w, v)$, find a cycle C that visits every vertex exactly once, minimizing the *maximum edge weight* in C . (In other words, our salesman doesn't care about the total travel time, but can't tolerate long flights and wants to keep each flight short.) Equivalently, we seek the smallest weight r^* such that $G[r^*]$ contains a Hamiltonian cycle, where $G[r]$ denotes the subgraph $(V, \{uv \in E \mid d(u, v) \leq r\})$.

Consider the following approach:

1. find the smallest r_0 such that $G[r_0]$ is connected
2. let T be any spanning tree of $G[r_0]$
3. get a (simple) cycle C from T and return C

To perform line 3, we pick an arbitrary node u as the root of T and call the following recursive procedure, which returns a simple path in G that visits all descendants of u :

- visit(u):
1. if u is a leaf then return $\langle u \rangle$
 2. let $u_1, \dots, u_k$ be the children of u
 3. for $i = 1, \dots, k$ do $\pi_i = \text{visit}(u_i)$
 4. return $\langle u \rangle \circ (\pi_1)^R \circ (\pi_2)^R \circ \dots \circ (\pi_k)^R$

Here, $\circ$ denotes concatenation, and π^R denotes the reverse of the list π . We get C by completing the resulting path into a cycle in G .

- (a) (70 pts) Show that the cycle C produced by the above procedure has maximum edge weight $\leq 3r_0$.

[Hint: You may want to first deduce some properties about the path generated by visit(). Try a few examples and see; then use induction to prove your observations formally.]

- (b) (30 pts) Conclude that there is a polynomial-time algorithm with approximation factor at most 3.

Solution:

(a) *Lemma.* For any node u , the path $P(u) = \langle p_0, p_1, \dots, p_m \rangle$ produced by `visit`(u) satisfies:

- (a) The last node p_m is one of the children of u .
- (b) The first edge (p_0, p_1) satisfies $d(u, p_1) \leq 2r_0$.
- (c) The closing edge (p_m, u) satisfies $d(p_m, u) \leq r_0$.

Proof. We prove by induction on the structure of T .

Base case: If u is a leaf, $P(u) = \langle u \rangle$, and all three properties hold trivially.

Inductive step: Suppose for each child u_i , the path $P(u_i)$ satisfies the lemma. Then:

- The last node p_m is the first node of $(P(u_k))^R$, which is u_k , a child of u .
- The first node after u is the last node of $(P(u_1))^R$, which is the last node of $P(u_1)$, say q_{m_1} . By inductive hypothesis, q_{m_1} is a descendant of u_1 . By the triangle inequality:

$$d(u, q_{m_1}) \leq d(u, u_1) + d(u_1, q_{m_1}) \leq r_0 + r_0 = 2r_0.$$

- The closing edge from p_m back to u connects u_k to u , and since u_k is a child of u , the edge has weight at most r_0 .

This completes the inductive proof of the lemma. □

Now consider the edges in the final cycle C :

- **Internal edges within each $(P(u_i))^R$:** These edges are either edges within a child subtree or its reverse. Since each subtree path corresponds to traversals in T , and every edge in T has weight at most r_0 , these internal edges are of weight at most r_0 .
- **Edges between subpaths:** When transitioning from the end of $(P(u_i))^R$ to the beginning of $(P(u_{i+1}))^R$, the two nodes involved are u_i (the child just completed) and some descendant $q_{m_{i+1}}$ of u_{i+1} . By the triangle inequality:

$$d(u_i, q_{m_{i+1}}) \leq d(u_i, u) + d(u, q_{m_{i+1}}).$$

Since u_i and u are connected by an edge in T of weight at most r_0 , and from the lemma $d(u, q_{m_{i+1}}) \leq 2r_0$, we get:

$$d(u_i, q_{m_{i+1}}) \leq r_0 + 2r_0 = 3r_0.$$

- **Closing edge:** Finally, the edge from p_m (which is u_k) back to u has weight at most r_0 by the lemma.

Thus, every edge in the constructed cycle C has weight at most $3r_0$.

(b) The algorithm provided in the question yields a polynomial-time, 3-approximation algorithm.

1. **Computing r_0 .** We sort all $\binom{n}{2}$ edge weights in non-decreasing order and scan them, maintaining the connectivity of the subgraph $G[r]$ using a Union-Find data structure. This allows us to determine the smallest r_0 such that $G[r_0]$ is connected. The time complexity of this step is $O(n^2 \log n)$.

2. **Building and traversing the spanning tree.** Once r_0 is known, we construct any spanning tree T of $G[r_0]$ using Prim's algorithm in $O(n^2)$ time. The recursive `visit` procedure then generates a path through all nodes in linear time relative to the number of vertices.
3. **Approximation.** As shown in (a), the final Hamiltonian cycle C has maximum edge weight at most $3r_0$. Since any valid Hamiltonian cycle in G must use only edges of weight $\leq r^*$, where r^* is the optimal value, and $r_0 \leq r^*$, we conclude:

$$\max_{e \in C} d(e) \leq 3r_0 \leq 3r^*.$$

Therefore, the algorithm runs in polynomial time and achieves a 3-approximation factor for the problem.

Problem 10.3: Given a *directed* graph $G = (V, E)$, we want to find a subgraph A that is acyclic (i.e., a dag), maximizing the number of edges in A .

(a) (50 pts) Show that the following deterministic algorithm yields a feasible solution with approximation factor at least $1/2$:

1. fix an arbitrary (not random) order of the vertices $v_1, \dots, v_n$
2. for $i = 2$ to n do {
3. IN_i = all edges from $\{v_1, \dots, v_{i-1}\}$ to v_i
4. OUT_i = all edges from v_i to $\{v_1, \dots, v_{i-1}\}$
5. if $|\text{IN}_i| \geq |\text{OUT}_i|$ then insert IN_i to A else insert OUT_i to A
- }

(b) (50 pts) Show that the following randomized algorithm yields a feasible solution and analyze its expected approximation factor:

1. take a random order of the vertices $v_1, \dots, v_n$
2. for each edge $(v_i, v_j) \in E$ do
3. if $i < j$ then insert (v_i, v_j) to A

[Note: it may be useful to recall the fact that a directed graph is a dag iff there exists a topological sort.]

Solution:

- (a) **Feasibility.** By construction, every edge inserted into A goes between v_i and some vertex in $\{v_1, \dots, v_{i-1}\}$. If we orient all chosen edges so that they go forward in the order $v_1 \prec v_2 \prec \dots \prec v_n$, then no edge ever points from a later vertex to an earlier one. Hence A admits the topological order $v_1, \dots, v_n$ and is acyclic.

1/2-Approximation. Let A be the edge set returned by the algorithm, and let E^* be an optimal maximum-cardinality acyclic subgraph of G .

Observe that every edge (u, v) with $u \neq v$ appears exactly once among the union of all $\text{IN}_i \cup \text{OUT}_i$, namely when the later of u, v is being processed. Hence

$$\sum_{i=2}^n (|\text{IN}_i| + |\text{OUT}_i|) = |E|.$$

By the choice in the algorithm, the edges added at step i is

$$\max\{|\text{IN}_i|, |\text{OUT}_i|\} \geq \frac{1}{2}(|\text{IN}_i| + |\text{OUT}_i|).$$

Summing over i gives

$$|A| = \sum_{i=2}^n \max\{|\text{IN}_i|, |\text{OUT}_i|\} \geq \frac{1}{2} \sum_{i=2}^n (|\text{IN}_i| + |\text{OUT}_i|) = \frac{1}{2} |E|.$$

Therefore

$$|A| \geq \frac{1}{2} |E| \geq \frac{1}{2} |E^*|,$$

as claimed.

- (b) **Feasibility.** Let π be the random order of vertices $(v_1, \dots, v_n)$. By construction, every edge in A goes forward in the random order π . Hence π itself is a valid topological sort of the subgraph (V, A) , so (V, A) is a DAG.

Expected approximation ratio. Let $E^* \subseteq E$ be an optimal maximum-cardinality acyclic subgraph, so $|E^*| = \max\{|A'| : (V, A') \text{ is a DAG}\}$. Note that trivially $|E^*| \leq |E|$. For any fixed edge $e = (u \rightarrow v) \in E$, under a uniformly random order π we have

$$\Pr[e \text{ is kept in } A] = \Pr(u \text{ precedes } v \text{ in } \pi) = \frac{1}{2}.$$

Define the indicator random variable

$$X_e = \begin{cases} 1, & \text{if } e \in A, \\ 0, & \text{otherwise.} \end{cases}$$

Then $\mathbb{E}[X_e] = \frac{1}{2}$, and linearity of expectation gives

$$\mathbb{E}[|A|] = \mathbb{E}\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} \mathbb{E}[X_e] = \sum_{e \in E} \frac{1}{2} = \frac{1}{2} |E|.$$

Since $|E| \geq |E^*|$, it follows that

$$\mathbb{E}[|A|] = \frac{1}{2} |E| \geq \frac{1}{2} |E^*|.$$

The expected approximation factor is $\frac{1}{2}$.